

Chapter 1 - Building Blocks (Questions and Answers)

Pregunta numero 1

1. Which of the following are legal entry point methods that can be run from the command line? (Choose all that apply.)
- A. `private static void main(String[] args)`
 - B. `public static final main(String[] args)`
 - C. `public void main(String[] args)`
 - D. `public static final void main(String[] args)`
 - E. `public static void main(String[] args)`
 - F. `public static main(String[] args)`

Answer 1: Lo primero que noto es que todas las opciones tienen el arreglo de tipo String, que representa los argumentos pasados por consola. El segundo punto que busco es que tengan el modificador de acceso public, ya que la JVM está fuera de la clase y necesita permiso para acceder al método; por eso debe ser public. El tercer punto que busco es que tenga la palabra reservada `static`, que indica que la JVM invoca este método directamente desde la clase, sin crear una instancia (objeto) de la misma previamente. El cuarto y último punto que busco es que retorne void, ya que el punto de entrada del programa no devuelve ningún valor a la JVM al terminar su ejecución. Por lo tanto, las respuestas correctas que contienen todos estos puntos son la D y la E. Es importante señalar que el modificador final es totalmente válido, ya que este modificador solo significa que no puede ser sobrescrito (overridden) por una subclase; esto no afecta en absoluto la capacidad de la JVM para encontrar y ejecutar el método estático.

Pregunta numero 2

2. Which answer options represent the order in which the following statements can be assembled into a program that will compile successfully? (Choose all that apply.)
- X: `class Rabbit {}`
Y: `import java.util.*;`
Z: `package animals;`
- A. X, Y, Z
 - B. Y, Z, X
 - C. Z, Y, X
 - D. Y, X
 - E. Z, X
 - F. X, Z
 - G. None of the above

Answer 2: Existe algo llamado la regla PIC; esta regla nos indica el orden de los elementos de un archivo Java. Debemos recordar que Java es sumamente estricto con el orden en el que se declaran los componentes principales dentro de un archivo .java. La primera letra del acrónimo PIC significa Package (Paquete); si lo llegamos a incluir, debe ser obligatoriamente la primera instrucción del archivo, sin contar líneas en blanco o comentarios; la segunda letra significa Import (Importaciones); estas se colocan si necesitamos importar librerías; deben ir después del package y antes de cualquier declaración de clase. Finalmente, la letra C significa Class (puede ser Clase, Interfaz o Enum). Tomando en cuenta todo eso, podemos decir que las respuestas correctas son los incisos C, D y E.

Pregunta numero 3

3. Which of the following are true? (Choose all that apply.)

```
public class Bunny {  
    public static void main(String[] x) {  
        Bunny bun = new Bunny();  
    } }
```

- A. Bunny is a class.
- B. bun is a class.
- C. main is a class.
- D. Bunny is a reference to an object.
- E. bun is a reference to an object.
- F. main is a reference to an object.
- G. The main() method doesn't run because the parameter name is incorrect.

Answer 3: En la primera línea de código tenemos que Bunny es una clase, por lo tanto, el inciso A es correcto; bun no es una clase, main no es una clase, es el método estático de entrada del programa, Bunny no es una referencia a un objeto, es una clase. Algo que es importante mencionar es que Java, por convención, indica que las clases deben empezar con letra mayúscula y que las variables/referencias deben empezar con minúscula. Finalmente, la otra respuesta correcta es el inciso E, ya que bun sí es una referencia a un nuevo objeto de la clase Bunny.

Pregunta numero 4

4. Which of the following are valid Java identifiers? (Choose all that apply.)

- A. `_`
- B. `_helloWorld$`
- C. `true`
- D. `java.lang`
- E. `Public`
- F. `1980_s`
- G. `_Q2_`

Answer 4: Existen 4 reglas estrictas referentes a los identificadores en Java. La primera es que un identificador solo puede contener letras, números, el símbolo de dólar (\$) y el guion bajo (_). El segundo punto es que al inicio debe comenzar obligatoriamente con una letra, un signo de dólar (\$) o un guion bajo (_), nunca puede empezar con un número. El tercer punto es que no podemos usar las palabras reservadas de Java (palabras como class, public, void, int, etc.) ni los valores literales (true, false o null). Finalmente, desde Java 9, un único guion bajo solitario (_) se considera una palabra clave y ya no puede usarse por sí solo como identificador. Por lo tanto, las respuestas correctas son los incisos B, E y G. Aquí es importante mencionar que la letra E es correcta porque Java es case-sensitive, lo que significa que Java distingue estrictamente entre mayúsculas y minúsculas. Teniendo presente esto, podemos decir que public y Public no son lo mismo; public es una keyword y Public se puede usar como identificador.

Pregunta numero 5

5. Which statements about the following program are correct? (Choose all that apply.)

```
2: public class Bear {  
3:     private Bear pandaBear;  
4:     private void roar(Bear b) {  
5:         System.out.println("Roar!");  
6:         pandaBear = b;  
7:     }  
8:     public static void main(String[] args) {  
9:         Bear brownBear = new Bear();  
10:        Bear polarBear = new Bear();  
11:        brownBear.roar(polarBear);  
12:        polarBear = null;  
13:        brownBear = null;  
14:        System.gc(); } }
```

- A. The object created on line 9 is eligible for garbage collection after line 13.
- B. The object created on line 9 is eligible for garbage collection after line 14.
- C. The object created on line 10 is eligible for garbage collection after line 12.
- D. The object created on line 10 is eligible for garbage collection after line 13.
- E. Garbage collection is guaranteed to run.
- F. Garbage collection might or might not run.
- G. The code does not compile.

Answer 5: Aquí el punto clave es el Garbage Collection; debemos recordar que un objeto se vuelve elegible para ser destruido por el Garbage Collector cuando ya no hay ninguna referencia activa que apunte a él. Si el programa ya no tiene forma de acceder al objeto en la memoria, Java lo marca como basura. Las respuestas correctas son los incisos A, D y F, ya que en la línea 12 se elimina una de las referencias al objeto 2; aún no era elegible para el Garbage Collector porque el objeto 1 todavía lo mantiene, pero después de la línea 13, se pierde el acceso al objeto 1. Entonces, como ya no podemos llegar al objeto 1, tampoco podemos llegar al objeto 2 que estaba guardado dentro de él, lo que quiere decir que ambos objetos quedan flotando sin referencia. Por último, la línea 14 que llama a [System.gc\(\)](#) es más una sugerencia o una solicitud para que el recolector de basura actúe, no es una orden a la JVM; por lo tanto, el Garbage Collection podría o no ejecutarse; debemos tener presente que la JVM tiene la última palabra y decidirá si ejecuta el recolector o lo ignora; nunca está garantizado.

Pregunta numero 6

6. Assuming the following class compiles, how many variables defined in the class or method are in scope on the line marked on line 14?

```
1: public class Camel {  
2:     { int hairs = 3_000_0; }  
3:     long water, air=2;  
4:     boolean twoHumps = true;  
5:     public void spit(float distance) {  
6:         var path = "";
```

56 Chapter 1 • Building Blocks

```
7:         { double teeth = 32 + distance++; }  
8:         while(water > 0) {  
9:             int age = twoHumps ? 1 : 2;  
10:            short i=-1;  
11:            for(i=0; i<10; i++) {  
12:                var Private = 2;  
13:            }  
14:            // SCOPE  
15:        }  
16:    }  
17: }
```

- A. 2
- B. 3
- C. 4
- D. 5
- E. 6
- F. 7
- G. None of the above

Answer 6: Aquí el punto clave es el alcance de las variables (scope). En Java, la regla general de oro es muy sencilla: una variable nace donde se declara y muere en la llave de cierre '}' del bloque en el que fue declarada. Entonces, la variable `water` es una variable de instancia; está viva en todo el objeto y puede usarse dentro de cualquier método no estático. La variable air es el mismo caso que la variable water, ya que fueron declaradas en la misma línea. La variable twoHumps

es una variable de instancia. La variable `distance` es el parámetro del método `spit`; su alcance abarca todo el cuerpo del método, o sea, hasta la línea 16. La variable `path` es una variable local del método; se declaró directamente en el bloque del método, por lo que vive hasta la línea 16. La variable `age` es una variable local declarada dentro del bloque del `while`, por lo tanto, su alcance abarca hasta que se cierra el `while` en la línea 15, lo que significa que en la línea 14 sigue viva la variable. Finalmente, la variable `i` es una variable local declarada también dentro del `while`. Aunque se usa dentro del ciclo `for` de la línea 11, fue declarada afuera de él. Su alcance termina en la línea 15, así que en la línea 14 sigue viva. Contando estas variables en alcance, nos da como respuesta el inciso F, o sea, 7 variables en el scope.

Pregunta numero 7

7. Which are true about this code? (Choose all that apply.)

```
public class KitchenSink {
    private int numForks;

    public static void main(String[] args) {
        int numKnives;
        System.out.print("""
            # forks = " + numForks +
            " # knives = " + numKnives +
            # cups = 0""");
    }
}
```

- A. The output includes: # forks = 0.
- B. The output includes: # knives = 0.
- C. The output includes: # cups = 0.
- D. The output includes a blank line.
- E. The output includes one or more lines that begin with whitespace.
- F. The code does not compile.

Answer 7: Aquí se están evaluando dos conceptos importantes, los Text Blocks y la interpolación de variables. Lo primero que debo mencionar es que no hay interpolación de variables en Java, lo que significa que Java no evalúa ni inyecta variables directamente dentro de un String, ni siquiera dentro de los Text Blocks. Todo lo que se coloque entre las triples comillas se considera texto literal, o sea, puros caracteres. En este ejercicio hay algunas trampas que debemos remarcar. Lo primero que debemos saber es que `numForks` es una variable de instancia, o sea que pertenece al objeto, y el método `main` es estático, o sea que pertenece a la

clase. Entonces podemos decir que acceder a numForks desde el main sin un objeto provocaría un error. Por otro lado, numKnives es una variable local que nunca fue inicializada. Si intentáramos usar su valor, el compilador arrojaría un error. El punto clave es que ninguna de estas reglas se rompe, ya que numForks y numKnives están dentro del Text Block, lo que significa que el compilador no las ve como variables; para él solo son letras. Esto quiere decir que, como nunca se intentan evaluar como variables, el código compila perfectamente sin errores. Entonces el texto se imprimirá tal cual está dentro de las triples comillas; con esto en mente, es fácil decir que la respuesta correcta es el inciso C.

Pregunta numero 8

8. Which of the following code snippets about var compile without issue when used in a method? (Choose all that apply.)
- A. `var spring = null;`
 - B. `var fall = "leaves";`
 - C. `var evening = 2; evening = null;`
 - D. `var night = Integer.valueOf(3);`
 - E. `var day = 1/0;`
 - F. `var winter = 12, cold;`
 - G. `var fall = 2, autumn = 2;`
 - H. `var morning = ""; morning = null;`

Answer 8: Cuando usamos ``var``, le estamos diciendo al compilador que debe “adivinar” el tipo de dato de la variable basándose en el valor que le estoy asignando. Para que esto funcione, existen restricciones estrictas: La primera restricción es que una variable declarada con ``var`` siempre debe ser inicializada en la misma línea. La segunda restricción es que no es posible asignarle un null de entrada porque null no tiene un tipo especificado (podría ser un string, un object, un animal, etc.). La tercera restricción es que no podemos declarar múltiples variables en la misma línea separadas por comas si usamos ``var``. La cuarta restricción es que, una vez que el compilador “adivina” el tipo en la primera línea, la variable se queda con ese tipo para siempre. Básicamente, no podemos asignarle valores incompatibles más adelante. Con eso en mente, podemos afirmar que las respuestas correctas son los incisos B, D, E y H.

Pregunta numero 9

9. Which of the following are correct? (Choose all that apply.)
- A. An instance variable of type `float` defaults to `0`.
 - B. An instance variable of type `char` defaults to `null`.
 - C. A local variable of type `double` defaults to `0.0`.
 - D. A local variable of type `int` defaults to `null`.
 - E. A class variable of type `String` defaults to `null`.
 - F. A class variable of type `String` defaults to the empty string `""`.
 - G. None of the above.

Answer 9: Existen dos reglas a la hora de hablar de los valores por defecto de las variables en Java. La primera es la regla del alcance (scope); esta nos indica que Java asigna automáticamente un valor por defecto si tú no las inicializas explícitamente, esto para las variables de instancia (de objeto) y para las variables de clase (estáticas). Para las variables locales (dentro de métodos), nunca reciben un valor por defecto; deben ser inicializadas explícitamente por nosotros antes de poder usarlas; de lo contrario, el código no compila. La segunda regla de oro es sobre los tipos de datos; por un lado, los primitivos numéricos (`byte`, `short`, `int`, `long`, `float`, `double`), por defecto son `0` o `0.00` para decimales; para los primitivos booleanos, por defecto es `false`; para los primitivos de tipo carácter (`char`), por defecto es el carácter nulo (`\u0000`), no la palabra `null`. Lo último que debemos tener en cuenta es que cualquier objeto o referencia, como `String`, arreglos o cualquier clase, por defecto, es `null`. Teniendo todo lo anterior presente, la respuesta correcta es el inciso E, ya que al ser una variable de clase (estática), recibe un valor por defecto. Justo como `String` es un objeto (una referencia), su valor por defecto es `null`.

Pregunta numero 10

10. Which of the following expressions, when inserted independently into the blank line, allow the code to compile? (Choose all that apply.)

```
public void printMagicData() {  
    var magic = _____;  
    System.out.println(magic);  
}
```

- A. 3_1
- B. 1_329_.0
- C. 3_13.0_
- D. 5_291._2
- E. 2_234.0_0
- F. 9___6
- G. _1_3_5_0

Answer 10: En esta pregunta evaluamos las reglas sobre el uso de guiones bajos (_) en literales numéricos, una característica que se introdujo en Java 7 para hacer que los números grandes sean más legibles. La regla de oro de los guiones bajos es que en Java podemos usar guiones bajos dentro de un número, siempre y cuando tengamos en mente que el guion bajo solo puede colocarse entre dos dígitos. De esta regla podemos derivar las siguientes prohibiciones: La primera es que no puede estar al principio del número. La segunda es que no puede estar al final del número. La tercera es que no puede estar pegado a un punto decimal (ni justo antes, ni justo después). La cuarta y última es que no puede estar pegado a los sufijos F, f, L, l, D o d. Teniendo lo anterior presente, podemos indicar que las respuestas correctas son los incisos A, E y F. Un punto importante a mencionar es que la respuesta de la letra F es válida porque podemos usar múltiples guiones bajos seguidos sin ningún problema, siempre y cuando estén flanqueados por dígitos en los extremos.

Pregunta numero 11

11. Given the following two class files, what is the maximum number of imports that can be removed and have the code still compile?

```
// Water.java
package aquarium;
public class Water { }
```

```
// Tank.java
package aquarium;
import java.lang.*;
import java.lang.System;
import aquarium.Water;
import aquarium.*;
public class Tank {
    public void print(Water water) {
        System.out.println(water); } }
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4
- F. Does not compile

Answer 11: En esta pregunta estamos viendo las importaciones redundantes (innecesarias) en Java. Hay dos reglas muy importantes que debemos tener en cuenta. Las reglas hablan sobre las importaciones automáticas: aquí nos dice que el paquete `java.lang` es “gratis”, o sea que Java importa automáticamente todas las clases del paquete `java.lang` en cada archivo que creas. Esto incluye clases fundamentales como `System`, `String`, `Integer`, `Math`, etc. El otro punto crucial es que debemos tener en cuenta que, si dos clases están dentro del mismo paquete (tienen la misma declaración `package`), se pueden ver y usar directamente entre sí. No necesitan importarse la una a la otra. Teniendo esto en mente, la respuesta correcta es el inciso E, ya que podemos eliminar todas las líneas que importan `java.lang` en el archivo `Tank`, ya que es innecesario importar `java.lang`; por otro lado, podemos

también eliminar las dos líneas que importan aquarium, ya que esta importación no se requiere, ya que tenemos el package aquarium.

Pregunta numero 12

12. Which statements about the following class are correct? (Choose all that apply.)

```
1: public class ClownFish {  
2:     int gills = 0, double weight=2;  
3:     { int fins = gills; }  
4:     void print(int length = 3) {  
5:         System.out.println(gills);  
6:         System.out.println(weight);  
7:         System.out.println(fins);  
8:         System.out.println(length);  
9:     } }
```

- A. Line 2 generates a compiler error.
- B. Line 3 generates a compiler error.
- C. Line 4 generates a compiler error.
- D. Line 7 generates a compiler error.
- E. The code prints 0.
- F. The code prints 2.0.
- G. The code prints 2.
- H. The code prints 3.

Answer 12: Hay varios errores que hay dentro del código; el primer error de compilación se encuentra en la línea 2, ya que en Java no podemos declarar e inicializar múltiples variables en una sola línea separada por comas si no son exactamente del mismo tipo. El siguiente error de compilación se encuentra en la línea 4, ya que se está intentando asignar un valor por defecto al parámetro length, ya que Java no soporta parámetros con valores por defecto en la firma de los métodos. El siguiente error de compilación lo encontramos en la línea 7; este es un problema de alcance (scope), ya que en dicha línea se está intentando usar la variable fins. Dicha variable nació y murió en la línea 3; por lo tanto, no es posible usarla en la línea 7. En resumen, las letras que tienen las respuestas correctas son los incisos A, C y D.

Pregunta numero 13

13. Given the following classes, which of the following snippets can independently be inserted in place of INSERT IMPORTS HERE and have the code compile? (Choose all that apply.)

```
package aquarium;  
public class Water {  
    boolean salty = false;  
}
```

```
package aquarium.jellies;  
public class Water {  
    boolean salty = true;  
}
```

```
package employee;  
INSERT IMPORTS HERE  
public class WaterFiller {  
    Water water;  
}
```

- A. `import aquarium.*;`
- B. `import aquarium.Water;`
`import aquarium.jellies.*;`
- C. `import aquarium.*;`
`import aquarium.jellies.Water;`
- D. `import aquarium.*;`
`import aquarium.jellies.*;`
- E. `import aquarium.Water;`
`import aquarium.jellies.Water;`
- F. None of these imports can make the code compile.

Answer 13: Aquí podemos ver los conflictos de nombres y cómo Java prioriza las importaciones. Debemos recordar que en Java, cuando tenemos dos clases con exactamente el mismo nombre (en este caso, `Water`) pero en diferentes paquetes (`aquarium` y `aquarium.jellies`), el compilador necesita saber de forma inequívoca a cuál nos estamos refiriendo cuando declaramos la variable `Water water`. Para esto tenemos dos reglas de oro para resolver los conflictos de importaciones. La primera es la importación explícita vs. comodín (wildcard `*`). Una importación explícita (nombrando la clase exacta) siempre le gana a una importación con comodín. Si el compilador encuentra ambas, ignorará la del comodín para esa clase específica. La segunda es el límite de la ambigüedad; si intentamos usar una clase y el compilador la encuentra en dos importaciones con comodín diferentes, se rinde y lanza un error de ambigüedad. Del mismo modo, está estrictamente prohibido hacer dos importaciones explícitas de clases con el mismo nombre. Con lo anterior en mente, podemos decir que las respuestas correctas son los incisos A, B y C. A es correcta porque solo estamos importando las clases del paquete `aquarium`. Cuando Java lee `Water water`, busca en el paquete importado, encuentra a `aquarium.Water` y lo

usa. Con esto no hay conflicto. La letra B es correcta gracias a la regla 1 que explicamos. Básicamente, cuando Java lee ``Water water``, la importación explícita de `import `aquarium.Water`` tiene prioridad sobre `import `aquarium.jellies.*``, lo que causa que compile sin problemas. Por último, la letra C es correcta debido al mismo caso de la letra B, pero a la inversa, o sea que la importación explícita de ``aquarium.jellies.Water`` le gana al comodín de `aquarium.*`

Pregunta numero 14

14. Which of the following statements about the code snippet are true? (Choose all that apply.)

```
3: short numPets = 5L;  
4: int numGrains = 2.0;  
5: String name = "Scruffy";  
6: int d = numPets.length();  
7: int e = numGrains.length;  
8: int f = name.length();
```

- A. Line 3 generates a compiler error.
- B. Line 4 generates a compiler error.
- C. Line 5 generates a compiler error.
- D. Line 6 generates a compiler error.
- E. Line 7 generates a compiler error.
- F. Line 8 generates a compiler error.

Answer 14: En esta pregunta, los temas principales son la conversión de tipos primitivos (casting) y la diferencia entre los primitivos y objetos. La primera línea que genera un error de compilación es la línea 2, esto debido a que el sufijo L indica que el número 5 es un literal de tipo long (64 bits). Esto quiere decir que estamos intentando meter un valor grande en una variable short, que es mucho más pequeña

(16 bits). Java no permite esta conversión de mayor a menor (narrowing) automáticamente para evitar pérdida de datos; para poder hacer esto, sería necesario realizar un cast explícito. La siguiente línea que genera un error de compilación es la línea 4; en esta línea el error es que estamos intentando asignar un valor double a una variable de tipo int; como se perderán los decimales, el compilador exige un cast explícito; sin este cast explícito, el código no va a compilar. El siguiente error de compilación lo encontramos en la línea 6; aquí la variable numPets fue declarada como un tipo primitivo. El problema radica en que los tipos primitivos no son objetos, por lo tanto, no tienen métodos integrados; entonces, al intentar llamar a .length() sobre un primitivo, nos provocará un error inmediato. Finalmente, en la línea 7 se genera un error de compilación; en esta línea ocurre exactamente lo mismo que en el caso anterior, ya que numGrains es un tipo primitivo, por lo que no tiene propiedades ni atributos; además, en Java el atributo .length es exclusivo de los arreglos, no de los primitivos numéricos. Con todo esto presente, podemos decir que las respuestas correctas son los incisos A, B, D y E.

Pregunta numero 15

15. Which of the following statements about garbage collection are correct? (Choose all that apply.)
- A. Calling `System.gc()` is guaranteed to free up memory by destroying objects eligible for garbage collection.
 - B. Garbage collection runs on a set schedule.
 - C. Garbage collection allows the JVM to reclaim memory for other objects.
 - D. Garbage collection runs when your program has used up half the available memory.
 - E. An object may be eligible for garbage collection but never removed from the heap.
 - F. An object is eligible for garbage collection once no references to it are accessible in the program.
 - G. Marking a variable `final` means its associated object will never be garbage collected.

Answer 15: Estamos de vuelta con el tema del Garbage Collector y debemos tener presente un concepto fundamental: el GC es autónomo e impredecible. Recordemos que el Garbage Collector de Java tiene un solo propósito: liberar memoria eliminando objetos inalcanzables. Ahora bien, nosotros, como programadores, no tenemos ningún control sobre cuándo o cómo hace su trabajo. Con esto presente, podemos decir que las respuestas correctas son los incisos C, E y F. Por un lado, el inciso C justo nos da la definición y el propósito fundamental del GC: recuperar la memoria que ocupan los objetos muertos para poder alojar nuevos objetos. Después tenemos el inciso E; aquí nos recuerda que, si nuestro programa tiene mucha memoria disponible, es posible que el GC decida no ejecutarse nunca para ahorrar recursos de procesamiento. Si el programa termina sin que el GC se haya ejecutado, esos objetos elegibles simplemente mueren junto con el programa, pero el GC como tal nunca los removió activamente durante la ejecución. Finalmente, en

el inciso F nos da justo la definición exacta de cuándo un objeto se convierte en basura y justo esto sucede cuando ya no hay forma de llegar a él a través del código vivo; eso vuelve al objeto elegible para ser eliminado como basura por el GC.

Pregunta numero 16

16. Which are true about this code? (Choose all that apply.)

```
var blocky = ""  
    squirrel \s  
    pigeon  \  
    termite"";  
System.out.print(blocky);
```

- A. It outputs two lines.
- B. It outputs three lines.
- C. It outputs four lines.
- D. There is one line with trailing whitespace.
- E. There are two lines with trailing whitespace.
- F. If we indented each line five characters, it would change the output.

Answer 16: Volvemos al tema de los Text Blocks; en este problema, el enfoque se encuentra específicamente en cómo el compilador maneja los espacios en blanco y los saltos de línea con las secuencias de escape modernas. Empecemos con el manejo de los saltos de línea. Por defecto, un Text Block respeta los saltos de línea tal y como los escribimos en el código. Sin embargo, si ponemos una barra invertida \ al final de una línea, le estamos indicando al compilador que ignore el salto de línea y concatene esa línea con la siguiente. Viendo el texto que está dentro del Text Block, podemos notar que la línea `squirrel \s` no termina con `\\`, por lo tanto, sí hay un salto de línea normal; sin embargo, en la línea siguiente, gracias a que termina con \, estamos suprimiendo el salto de línea, lo que significa que esta línea se va a concatenar con la línea siguiente, lo que nos da como resultado que el Text Block solo tenga dos líneas de texto reales al imprimirse. Existe otra regla de oro de los Text Blocks; esta regla nos indica el manejo de los espacios al final (secuencia \s). La regla nos dice que los Text Blocks eliminan automáticamente todos los espacios en blanco finales de cada línea. Si nosotros queremos conservar un espacio al final de nuestro texto, debemos usar la secuencia de escape \s, como en la línea que dice squirrel \s; esto obliga a Java a preservar ese espacio al final de la palabra. Con esto en mente, podemos afirmar que las respuestas correctas son los incisos A y D.

Pregunta numero 17

17. What lines are printed by the following program? (Choose all that apply.)

```
1: public class WaterBottle {  
2:     private String brand;  
3:     private boolean empty;  
4:     public static float code;  
5:     public static void main(String[] args) {  
6:         WaterBottle wb = new WaterBottle();
```

Review Questions

```
7:         System.out.println("Empty = " + wb.empty);  
8:         System.out.println("Brand = " + wb.brand);  
9:         System.out.println("Code = " + code);  
10:    } }
```

- A. Line 8 generates a compiler error.
- B. Line 9 generates a compiler error.
- C. Empty =
- D. Empty = false
- E. Brand =
- F. Brand = null
- G. Code = 0.0
- H. Code = 0f

Answer 17: En esta pregunta estamos evaluando los valores por defecto de las variables de instancia y de clase, y sobre los modificadores de acceso. Lo primero que debemos notar es que ninguna de las variables fue inicializada explícitamente; por lo tanto, Java les asigna sus valores por defecto cuando se crea el objeto y cuando se carga la clase. Empezando por la variable `empty`, es un primitivo boolean, así que el valor por defecto es `false` y ese va a ser el valor que se imprime en la línea 7 (`Empty = false`). Después, la variable `brand` es de tipo `String` (un objeto/referencia), por lo que su valor por defecto siempre es `null` y ese va a ser el

valor que se imprime en la línea 8 (Brand = null). Finalmente, la variable code es un primitivo float, por lo que su valor por defecto es 0.0 y esto causará que la línea 9 imprima Code = 0.0. Esto nos da como resultado que las respuestas correctas sean los incisos D, F y G.

Pregunta numero 18

18. Which of the following statements about `var` are true? (Choose all that apply.)

- A.** A `var` can be used as a constructor parameter.
- B.** The type of a `var` is known at compile time.
- C.** A `var` cannot be used as an instance variable.
- D.** A `var` can be used in a multiple variable assignment statement.
- E.** The value of a `var` cannot change at runtime.
- F.** The type of a `var` cannot change at runtime.
- G.** The word `var` is a reserved word in Java.

Answer 18: En esta pregunta estamos evaluando las limitaciones de la inferencia de tipos con `var`. Recordemos que la palabra `var` le dice al compilador de Java que averigüe el tipo de dato basándose en el valor que le estoy asignando a la variable. Es importante recordar que Java sigue siendo un lenguaje de tipado fuerte y estático; debido a esto, debemos pensar en `var` como un atajo visual para no escribir el tipo explícitamente, pero el compilador hace el trabajo de asignarle un tipo inmutable antes de ejecutar el programa. Lo siguiente que debemos saber es que la inferencia de tipos la realiza el compilador de Java en tiempo de compilación, no la JVM en tiempo de ejecución. El siguiente punto que debemos saber es que el uso de `var` está estrictamente limitado a variables declaradas dentro de métodos, constructores o bloques de inicialización. No podemos usar `var` para definir atributos de una clase. El último punto crucial es que, una vez que el compilador deduce que un `var` es, por ejemplo, de tipo `String`, esa variable será `String` para siempre. No podemos reasignarle otro tipo, como uno numérico o un booleano, más adelante. Con esto podemos afirmar que las respuestas correctas son los incisos B, C y F.

Pregunta numero 19

19. Which are true about the following code? (Choose all that apply.)

```
var num1 = Long.parseLong("100");  
var num2 = Long.valueOf("100");  
System.out.println(Long.max(num1, num2));
```

- A. The output is 100.
- B. The output is 200.
- C. The code does not compile.
- D. num1 is a primitive.
- E. num2 is a primitive.

Answer 19: En esta pregunta se está evaluando un concepto muy específico, que son las clases envoltantes (wrapped classes) y la diferencia exacta entre los métodos `parse...()` y `valueOf()`. Empecemos por saber que en todas las clases numéricas de Java (Integer, Long, Double, etc.), existen dos métodos para convertir un String en un número, pero con una diferencia clave en lo que devuelven. Por un lado, los métodos que empiezan con `parse` como `parseLong` o `parseInt`, siempre retornan un tipo primitivo (long, int), mientras que los métodos que empiezan con `valueOf` siempre retornan un objeto de la clase envoltante (Long, Integer). Con esto en mente, tenemos que las respuestas correctas son los incisos A y D. Cuando se usó `parseLong`, el resultado es primitivo. El compilador infiere que la variable `num1` es del tipo primitivo long. Donde se usa el `valueOf`, el resultado es un objeto; el compilador infiere que la variable `num2` es de la clase Long. Debemos también recordar que el método `Long.max` compara dos números para devolver el mayor. Aunque `num2` es un objeto y no un primitivo, Java aplica auto-unboxing, lo que extrae el valor primitivo del objeto Long de forma transparente y automática para poder hacer la operación. Otro punto a destacar es que, tal como establece la regla, `parseLong` obliga a `var` a inferir un tipo primitivo long, por lo que la variable `num1` es primitiva. Con esto en mente, podemos decir que las respuestas correctas son los incisos A y D.

Pregunta numero 20

20. Which statements about the following class are correct? (Choose all that apply.)

```
1: public class PoliceBox {  
2:     String color;  
3:     long age;  
4:     public void PoliceBox() {  
5:         color = "blue";  
6:         age = 1200;  
7:     }  
8: }
```

62 Chapter 1 • Building Blocks

```
9:     public static void main(String []time) {  
10:         var p = new PoliceBox();  
11:         var q = new PoliceBox();  
12:         p.color = "green";  
13:         p.age = 1400;  
14:         p = q;  
15:         System.out.println("Q1="+q.color);  
16:         System.out.println("Q2="+q.age);  
17:         System.out.println("P1="+p.color);  
18:         System.out.println("P2="+p.age);  
19:     } }
```

- A. It prints Q1=blue.
- B. It prints Q2=1200.
- C. It prints P1=null.
- D. It prints P2=1400.
- E. Line 4 does not compile.
- F. Line 12 does not compile.
- G. Line 13 does not compile.
- H. None of the above.

Answer 20: En esta pregunta hay una trampa común; esta es la trampa del falso constructor. En la línea 4 tenemos dicho falso constructor de la clase, pero el problema es que en Java los constructores no tienen tipo de retorno, ni siquiera void; por lo tanto, al agregar la palabra void, el compilador de Java lo interpreta como un método ordinario que en este caso se llama igual que la clase. Cabe aclarar que, aunque es una pésima práctica nombrar un método igual que la clase, es sintaxis legal en Java y compila perfectamente. Entonces, como no definimos un constructor real, Java proporciona el constructor por defecto. Vacío, esto significa que cuando creamos un objeto PoliceBox, sus variables de instancia se inicializan con sus valores por defecto, o sea que la variable color será null por defecto y la variable age será 0 por defecto. Esto sucede, ya que el código dentro de ese método “falso” nunca se va a ejecutar automáticamente. Siguiendo línea por línea el seguimiento de referencias en memoria, podemos afirmar que la respuesta correcta es el inciso C, ya que en la línea 16 se imprime p.color; como p apunta al objeto 2, su color será null. Esto se observa al ver que tanto p como q apuntan al objeto 2, el

cual nunca fue modificado y, debido a esto, conserva sus valores por defecto, los cuales son color = null y age = 0.

Pregunta numero 21

21. What is the output of executing the following class?

```
1: public class Salmon {  
2:     int count;  
3:     { System.out.print(count+"-"); }  
4:     { count++; }  
5:     public Salmon() {  
6:         count = 4;  
7:         System.out.print(2+"-");  
8:     }  
9:     public static void main(String[] args) {  
10:        System.out.print(7+"-");  
11:        var s = new Salmon();  
12:        System.out.print(s.count+"-"); } }
```

Review Questions

- A. 7-0-2-1-
- B. 7-0-1-
- C. 0-7-2-1-
- D. 7-0-2-4-
- E. 0-7-1-
- F. The class does not compile because of line 3.
- G. The class does not compile because of line 4.
- H. None of the above.

Answer 21: En esta pregunta estamos evaluando el orden de inicialización en Java. Aquí la clave es memorizar el orden exacto en el que Java ejecuta las cosas cuando creamos un objeto. Lo primero que se ejecuta son los bloques estáticos (static blocks); estos se ejecutan una sola vez cuando la clase se carga. Lo segundo que se ejecuta es el método main; recordemos que este es el punto de entrada de la aplicación. El tercero son las variables de instancia y bloques de inicialización de instancia; estos se ejecutan en el orden exacto en el que aparecen escritos en el

archivo, de arriba hacia abajo. Por último, tenemos el constructor; siempre se ejecuta al final de la inicialización de la instancia. Con esto en mente y siguiendo el flujo del programa, que básicamente es que el programa arranca en el método main, se imprime directamente el texto 7-, de ahí se invoca la creación del objeto y Java pausa el main para construirlo, se declara count; al no asignarle valor, toma su valor por defecto, que es 0, de ahí se imprime count + "-" y, ya que count vale 0, se imprime 0-. Hasta este momento tenemos 7-0-. Lo anterior es del primer bloque de inicialización; al pasar al segundo bloque de inicialización, se ejecuta el count++ y con esto la variable count ahora vale 1. Lo siguiente en entrar es el constructor; de ahí se reasigna el valor de la variable, por lo tanto, count = 4. De ahí se imprime 2- en la línea 7; con esto ya tenemos 7-0-2-. Finalmente, regresamos al main; se pide imprimir el s.count + "-". Como el constructor dejó el valor en 4, esto imprime 4-, dándonos como resultado final 7-0-2-4- lo que nos permite asegurar que la respuesta correcta es el inciso D.

Pregunta numero 22

22. Given the following class, which of the following lines of code can independently replace INSERT CODE HERE to make the code compile? (Choose all that apply.)

```
public class Price {  
    public void admission() {  
        INSERT CODE HERE  
        System.out.print(amount);  
    } }  

```

- A. `int Amount = 0b11;`
- B. `int amount = 9L;`
- C. `int amount = 0xE;`
- D. `int amount = 1_2.0;`
- E. `double amount = 1_0_.0;`
- F. `int amount = 0b101;`
- G. `double amount = 9_2.1_2;`
- H. `double amount = 1_2_.0_0;`

Answer 22: Esta pregunta puede funcionar como un repaso de varios conceptos; por un lado, tenemos el case-sensitivity, la conversión de tipos (casting) y las reglas de los guiones bajos y prefijos en literales numéricos. El primer inciso correcto es la C; esta respuesta es válida, ya que el prefijo 0x indica que el número está en sistema hexadecimal, la letra E, que vale 14 en decimal, es un dígito hexadecimal válido y cabe perfectamente dentro de un int. De ahí saltamos al inciso F; este también es válido. El prefijo 0b indica que es un numero binario; 101, que equivale a 5 en decimal, es válido y cabe perfectamente en un int. Finalmente, llegamos al inciso G, el cual también es válido; es un numero decimal de tipo double, donde los guiones bajos están colocados estrictamente entre dos dígitos, cumpliendo a la

perfección con la regla de los guiones bajos. Con lo anterior dicho, podemos afirmar que las respuestas correctas son los incisos C, F y G.

Pregunta numero 23

23. Which statements about the following class are true? (Choose all that apply.)

```
1: public class River {  
2:     int Depth = 1;  
3:     float temp = 50.0;  
4:     public void flow() {  
5:         for (int i = 0; i < 1; i++) {  
6:             int depth = 2;  
7:             depth++;  
8:             temp--;  
9:         }  
}
```



```
10:     System.out.println(depth);  
11:     System.out.println(temp); }  
12: public static void main(String... s) {  
13:     new River().flow();  
14: } }
```

- A. Line 3 generates a compiler error.
- B. Line 6 generates a compiler error.
- C. Line 7 generates a compiler error.
- D. Line 10 generates a compiler error.
- E. The program prints 3 on line 10.
- F. The program prints 4 on line 10.
- G. The program prints 50.0 on line 11.
- H. The program prints 49.0 on line 11.

Answer 23: Aquí estamos evaluando los tipos de datos primitivos por defecto y el alcance de las variables (scope) junto con el case-sensitive. El primer error lo encontramos en la línea 3; debemos recordar que Java, con cualquier número literal con punto decimal, es considerado por defecto de tipo double, que ocupa 64 bits de memoria. La variable temp fue declarada como float, que ocupa 32 bits de memoria, por lo que es más pequeña. El compilador prohíbe intentar meter un double dentro de un float de forma directa porque se podría perder precisión. Para que esta línea compilara sin ningún problema, tendría que llevar la letra f al final de valor o hacer un cast explícito al tipo float; al no tener ninguno de los dos, nos genera un error de compilación. El otro error de compilación lo encontramos en la línea 10. Recordemos que en la línea 2 declaramos una variable de instancia llamada Depth (con D mayúscula); después, en la línea 6, dentro del ciclo for, declaramos una variable local llamada depth (con d minúscula). Como Java distingue entre mayúsculas y minúsculas, no hay conflicto de nombres; para Java son dos variables distintas. Hasta este punto no tenemos ningún problema; sin embargo, en la línea 9 ocurre un problema cuando se cierra la llave } del ciclo for. En ese preciso instante, la variable local `depth` muere porque sale de su alcance, lo que causa que cuando se intenta imprimir la variable `depth` en la línea 10, el compilador va a lanzar un error porque para él esa variable ya no existe. Cabe mencionar que si hubiéramos impreso la variable `Depth`, la línea 10 sí habría compilado porque habría usado la variable de la línea 2. Ya con lo anterior plasmado, podemos afirmar que las respuestas correctas son los incisos A y D.